11.Linked Lists

Una linked list è un collezione di nodi con vari campi , dove contiene i campi dati e i campi link
La caratteristica principale di una lista semplicemente collegata è che il collegamento avviene in una sola direzione: ogni nodo punta solo al successivo, e non esiste un riferimento al nodo precedente.

Perchè usare le liste? (confronto con gli array)

È importante capire perché si usano rispetto agli array:

- **Problemi degli Array:** Hanno dimensione fissa, richiedono memoria contigua e l'inserimento/cancellazione in posizioni intermedie è complesso (richiede lo spostamento degli elementi).
- **Vantaggi delle Liste:** Dimensione dinamica, allocazione sparsa in memoria, inserimenti e cancellazioni sono più semplici e veloci.

REPRESENTATION AND OPERATIONS OF SINGLY

- rappresentazione di un nodo
  1. **Dati (data):** Contengono il valore o le informazioni che il nodo rappresenta.
  2. **Puntatore (next):** È un riferimento o un indirizzo che punta al prossimo nodo della lista.
- **Strutture di Supporto (Side Structures)** Per gestire la lista servono puntatori esterni ai nodi stessi:
  1. **pFirst (Head):** Punta sempre al primo elemento della lista.
  2. **pLast (Tail - Opzionale):** Punta all'ultimo elemento. È utile per velocizzare l'inserimento in coda senza dover scansionare tutta la lista.
- L'allocazione nell'heap
  1. Avviene prima dichiarando il puntatore
  2. E successivamente viene eseguita l'allocazione nella memoria dinamica per un singolo oggetto del tipo scelto `malloc(sizeof(tipo))`
- Inizializzazione :
  1. Dopo aver allocato , si inizializzano i membri presenti dentro l'oggetto
  2. E inizializzare il puntatore a NULL `p->pNext = NULL;`

Operazioni Principali (Logica)

Le operazioni sulle liste richiedono attenzione alla gestione dei puntatori per non perdere dati o rompere la catena

1. Scansione (Print)

Non si usa **`pFirst`** per muoversi, ma si crea un puntatore temporaneo **`pScan`** inizializzato a `**pFirst**` e si avanza finche `pScan != NULL`

2. Inserimento in testa (Head insertion)

Ordine cruciale :

- Creare il nuovo nodo (***pNew***)
- Far puntatore ***pNew*** -> *pNext* l'attuale ***pFirst***
- Aggiornare ***pFirst*** facendolo a puntare ***pNew***

3. Cancellazione in testa (Head deletion)ù

Bisogna salvare il riferimento al secondo nodo prima di cancellare il primo:

- creare un puntatore ***temp*** che punta al secondo nodo (pFirst -> pNext)
- Liberare la memoria del primo nodo con ***free(pFirst***)
- Aggiornare *pFirst* assegnandogli *temp*

Circular singly linked list

Questa è simila alla prima ma con una differenza fondamentale , l'ultimo nodo non punta a `NULL` ma al primo nodo della lista

```
Nodo1 -> Nodo2 -> Nodo3 -> Nodo1 (Ciclico)
```

Doubly linked list

Una Lista doppiamente collegata è una struttura di dati in cui ogni nodo contiene 3 parti :

1. Un dato
2. un puntatore al nodo successivo come nella lista singolarmente collegata
3. un puntatore al nodo precedente che permette di navigare indietro alla lista

```
NULL <- Nodo1 <-> Nodo2 <-> Nodo3 -> NULL
```